

数据库的完整性是指数据库数据的正确性 (correctness) 和相容性 (compatibility)。数据的正确性是指数据库数据符合现实世界语义且反映当前的实际状况。数据的相容性是指数据库同一对象在不同关系表中的数据是相同的, 一致的。

本章导读

本章系统地讲解关系数据库管理系统完整性实现的机制, 包括完整性约束的定义机制、检查方法和违约处理方法等。

实体完整性和参照完整性是关系系统的两个不变性, 即关系模型必须具备的完整性约束。其他完整性约束则可以归入用户定义的完整性, 由现实世界语义约束确定。

完整性约束作为数据库模式的一部分存入数据字典, 在数据库数据被修改时, 关系数据库管理系统的完整性约束检查机制将按照数据字典中定义的这些约束进行检查和处理。数据库完整性的定义可由 SQL 的数据定义语言 (DDL) 实现。

本章还讲解了触发器 (trigger), 触发器可以实施更为复杂的完整性定义、检查和违约操作, 具有更精细和更强大的数据控制能力。因为触发器规则中的动作体可以很复杂, 通常是一段过程化 SQL。

读者要通过上机实验掌握 SQL 的完整性约束定义功能、属性和元组完整性约束的定义语句, 能够验证完整性约束检查机制是否发挥作用, 掌握违约处理方法。进一步地, 要掌握触发器的概念, 并能使用触发器实现更复杂的完整性控制方法。

5.1 数据库完整性概述

数据库的完整性是指数据库数据的正确性和相容性。例如, 学生的学号必须唯一, 学生的性别只能是男或女, 百分制的课程成绩取值范围为 0~100, 学生所选的课程必须是学校开设的课程, 学生所在的院系必须是学校已成立的院系等。

数据库的完整性和数据库的安全性是两个既有联系又不尽相同的概念。

数据库的完整性是防止数据库中存在不符合语义的数据,也就是防止数据库中存在不正确的数据。数据的安全性是保护数据库防止恶意的破坏和非法的存取。因此,完整性检查和控制的防范对象是不合语义的、不正确的数据,防止它们进入数据库。安全性控制的防范对象是非法用户和非法操作,防止他们对数据库数据的非法存取。

为维护数据库的完整性,关系数据库管理系统必须能够实现以下功能。

1. 提供定义完整性约束的机制

完整性约束也称为完整性规则,是数据库中的数据必须满足的语义约束。它表达了给定的数据模型中数据及其联系所具有的制约和依存规则,用以限定符合数据模型的数据库状态以及状态的变化,以保证数据的正确、有效和相容。SQL 标准使用了一系列概念来描述完整性约束,包括关系模型的实体完整性、参照完整性和用户定义的完整性。这些完整性约束一般由 SQL 的数据定义语言语句来实现。它们作为数据库模式的一部分存入数据字典。

2. 提供检查完整性约束的方法

关系数据库管理系统中检查数据是否满足完整性约束的机制称为完整性检查。一般在 INSERT、UPDATE、DELETE 语句执行后开始检查,也可以在事务提交时检查。检查这些操作执行后数据库中的数据是否违背了完整性约束。

3. 提供完整性的违约处理方法

关系数据库管理系统若发现用户的操作违背了完整性约束,就采取一定的动作,如拒绝 (NO ACTION) 执行该操作或级联 (CASCADE) 执行其他操作,进行违约处理以保证数据的完整性。

早期的关系数据库管理系统不支持完整性约束检查,因为该项检查费时费资源。现在商用的关系数据库管理系统产品都支持完整性控制,不必由应用程序来完成,从而减轻了应用程序员的负担。

更重要的是,完整性控制已成为关系数据库管理系统核心支持的功能,从而能够为所有用户和应用提供一致的数据库完整性。由应用程序来实现完整性控制是有漏洞的,有的应用程序定义的完整性约束可能会被其他应用程序破坏,因此数据库数据的正确性仍然无法保障。

本书第2章2.3节已讲解了关系数据库三类完整性约束的基本概念,下面将介绍用 SQL 实现这些完整性控制功能的方法。

5.2 实体完整性

5.2.1 定义实体完整性

关系模型的实体完整性在 CREATE TABLE 中用 PRIMARY KEY 定义。对单属性构成的码有两种说明方法,一种是定义为列级约束,另一种是定义为表级约束。对多个属性构成的码只

有一种说明方法,即定义为表级约束。

[例 5.1] 创建“学生”表 Student, 将 Sno 属性定义为主码。

```
CREATE TABLE Student
( Sno CHAR(8) PRIMARY KEY,           /* 在列级定义主码 */
  Sname CHAR(20) UNIQUE,
  Ssex CHAR(6),
  Sbirthdate Date,
  Smajor VARCHAR(40)
);
```

或者

```
CREATE TABLE Student
( Sno CHAR(8),
  Sname CHAR(20) UNIQUE,
  Ssex CHAR(6),
  Sbirthdate Date,
  Smajor VARCHAR(40),
  PRIMARY KEY(Sno)                   /* 在表级定义主码 */
);
```

[例 5.2] 创建“学生选课”表 SC, 将(Sno,Cno)属性组定义为主码。

```
CREATE TABLE SC
( Sno CHAR(8),
  Cno CHAR(5),
  Grade SMALLINT,
  Semester CHAR(5),                 /* 开课学期 */
  Teachingclass CHAR(8),           /* 学生选修某一门课所在的教学班 */
  PRIMARY KEY(Sno, Cno)             /* 主码由两个属性构成,必须在表级定义主码 */
);
```

5.2.2 实体完整性检查和违约处理

用 PRIMARY KEY 短语定义了关系的主码后,每当用户程序对基本表插入一条记录或对主码列进行更新操作时,关系数据库管理系统将按照实体完整性规则自动进行检查。包括:

- ① 检查主码值是否唯一,如果不唯一则拒绝插入或修改。
- ② 检查主码的各个属性是否为空,只要有一个为空就拒绝插入或修改。

从而保证了实体完整性。

检查记录中主码值是否唯一的一种方法是进行全表扫描,依次判断表中每一条记录的主码值与将插入记录的主码值(或修改的新主码值)是否相同,如图 5.1 所示。



图 5.1 用全表扫描方法检查主码唯一性

全表扫描是十分耗时的。为了避免对基本表进行全表扫描,关系数据库管理系统一般都在主码上自动建立一个索引,如图 5.2 所示的 $B+$ 树索引,通过索引查找基本表中是否已经存在新的主码值将大大提高效率。例如,如果新插入记录的主码值是 25,通过主码索引,从 $B+$ 树的根结点开始查找,只要读取三个结点就可以知道该主码值已经存在,即根结点(51)、中间结点(12 30)和叶结点(15 20 25)。所以不能插入这条记录。如果新插入记录的主码值是 86,也只要查找三个结点就可以知道该主码值不存在,所以可以插入该记录。

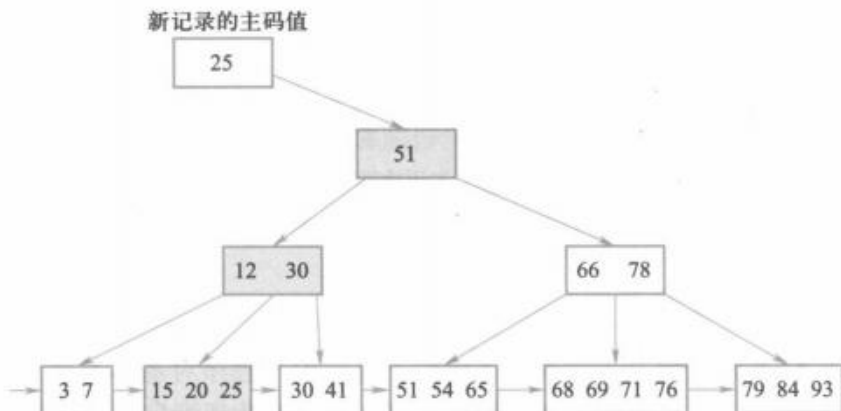


图 5.2 使用索引检查主码唯一性示例

5.3 参照完整性

5.3.1 定义参照完整性

关系模型的参照完整性在 CREATE TABLE 中用 FOREIGN KEY 短语定义哪些列为外码, 用 REFERENCES 短语指明这些外码参照哪些表的主码。

例如, 关系 SC 中一个元组表示一个学生选修某门课程的成绩, (Sno, Cno) 是主码。Sno、Cno 分别参照引用 Student 表的主码和 Course 表的主码。

[例 5.3] 定义 SC 中的参照完整性。

```
CREATE TABLE SC
(Sno CHAR(8),
Cno CHAR(5),
Grade SMALLINT,
Semester CHAR(5),
Teachingclass CHAR(8),
PRIMARY KEY(Sno, Cno),      /* 在表级定义实体完整性 */
FOREIGN KEY(Sno) REFERENCES Student(Sno),
/* 在表级定义参照完整性, Sno 是外码, 被参照表是 Student */
FOREIGN KEY(Cno) REFERENCES Course(Cno)
/* 在表级定义参照完整性, Cno 是外码, 被参照表是 Course */
);
```

5.3.2 参照完整性检查和违约处理

参照完整性将两个表中的相应元组联系起来了。因此, 对被参照表和参照表进行更新(增、删、改)操作时有可能破坏参照完整性, 必须进行检查以保证这两个表的相容性。

例如, 对表 SC 和 Student 有 4 种可能破坏参照完整性的情况及违约处理, 如表 5.1 所示。

表 5.1 可能破坏参照完整性的情况及违约处理

被参照表 (如 Student)	参照表 (如 SC)	违约处理
可能破坏参照完整性	插入元组	拒绝
可能破坏参照完整性	修改外码值	拒绝
删除元组	可能破坏参照完整性	拒绝/级联删除/设置为空值
修改主码值	可能破坏参照完整性	拒绝/级联修改/设置为空值

① SC 表中增加一个元组, 该元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

② 修改 SC 表中的一个元组, 修改后该元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

③ 从 Student 表中删除一个元组, 造成 SC 表中某些元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

④ 修改 Student 表中一个元组的 Sno 属性, 造成 SC 表中某些元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

当上述不一致发生时, 系统可以采用以下策略加以处理。

① 拒绝执行。不允许该操作执行。该策略一般设置为默认策略。

② 级联操作。当删除或修改被参照表(Student)的一个元组导致与参照表(SC)的不一致时, 删除或修改参照表中所有导致不一致的元组。

例如, 删除 Student 表中 Sno 值为“20180001”的元组, 则要从 SC 表中级联删除“SC. Sno = '20180001'”的所有元组。

③ 设置为空值。当删除或修改被参照表的一个元组时造成不一致, 则将参照表中所有造成不一致的元组的对应属性设置为空值。例如, 下面两个关系:

学生(学号, 姓名, 性别, 出生日期, 主修专业)

专业(专业名, 专业编码)

其中“学生”关系的“主修专业”是外码, 参照“专业”关系的主码“专业名”。

假设“专业”表中“专业名”为计算机科学与技术的元组被删除, 按照表 5.1 的策略, 就要把“学生”表中“主修专业 = ‘计算机科学与技术’”的所有元组的主修专业设置为空值。这样处理的语义可以解释为: 某个专业删除了, 则选择该专业为主修专业的所有学生就要等待重新选择主修专业。

这里讲解一下外码能否接受空值的问题。

例如, “学生”表中“主修专业”是外码, 按照应用的实际情况可以取空值, 表示这个学生的主修专业尚未确定。

但在“学生选课”数据库中, 关系 SC 的 Sno 和 Cno 为外码, 它能否取空值呢? 答案是“不能”。因为 Sno 和 Cno 是关系 SC 的主属性, 按照实体完整性约束条件, 主属性不能为空值。若 SC 的 Sno 为空值, 则表明尚不存在的某个学生或者某个不知学号的学生选修了某门课程, 其成绩记录在 Grade 列中。这与学校的应用环境是不相符的, 因此 SC 的 Sno 列不能取空值。同样, SC 的 Cno 列也不能取空值。

因此对于参照完整性, 除了应该定义外码, 还应定义外码列是否允许空值。

一般地, 当对参照表和被参照表的操作违反了参照完整性时, 系统选用默认策略, 即拒绝执行。如果想让系统采用其他策略, 则必须在创建参照表时显式地加以说明。

[例 5.4] 显式说明参照完整性的违约处理示例。

```
CREATE TABLE SC
(
    Sno CHAR(8),
    Cno CHAR(5),
    Grade SMALLINT,          /* 成绩 */
    Semester CHAR(5),        /* 开课学期 */
    Teachingclass CHAR(8),    /* 学生选修某一门课所在的教学班 */
    PRIMARY KEY(Sno,Cno), /* 在表级定义实体完整性,Sno、Cno 都不能取空值 */
    FOREIGN KEY(Sno) REFERENCES Student(Sno) /* 在表级定义参照完整性 */
    ON DELETE CASCADE /* 当删除 Student 表中的元组时, */
                        /* 级联删除 SC 表中 Sno 与 Student.Sno 相同的 SC 元组 */
    ON UPDATE CASCADE, /* 当更新 Student 表中的 Sno 时, */
                        /* 级联更新 SC 表中相应元组的 Sno */
    FOREIGN KEY(Cno) REFERENCES Course(Cno)
                        /* 在表级定义参照完整性 */
    ON DELETE NO ACTION /* 当删除 Course 表中的元组造成与 SC 表不一致,即 SC 中存在
                        /* 这样的元组,其 Cno 与 Course.Cno 相同,则拒绝删除 Course
                        /* 表的这个元组 */
    ON UPDATE CASCADE /* 当更新 Course 表中的 Cno 时, */
                        /* 级联更新 SC 表中相应元组的 Cno */
);
```

可以对 DELETE 和 UPDATE 采用不同的策略。例如,例 5.4 中当删除被参照表 Course 表中的元组,造成与参照表(SC 表)不一致时,拒绝删除被参照表的元组;对更新操作则采用级联更新的策略。

从上面的讨论可以看到,关系数据库管理系统在实现参照完整性时,除了要提供定义主码、外码的机制外,还需要提供不同的策略供用户选择。具体选择哪种策略,要根据应用环境的要求确定。

5.4 用户定义的完整性

用户定义的完整性就是某一具体应用涉及的数据必须满足的语义要求。目前的关系数据库管理系统都提供了定义和检查这类完整性的机制,使用和实体完整性、参照完整性相同的技术和方法来处理它们,而不必由应用程序实现这一功能。

5.4.1 属性上的约束

1. 属性上约束的定义

在 CREATE TABLE 中定义属性的同时,可以根据应用要求定义属性上的约束,即属性值限制,包括:

- ① 列值非空(NOT NULL)。
- ② 列值唯一(UNIQUE)。
- ③ 检查列值是否满足一个条件表达式(CHECK 短语)。

2. 不允许取空值

[例 5.5]在定义 SC 表时,说明 Sno、Cno、Grade 属性不允许取空值。

```
CREATE TABLE SC
(Sno CHAR(8) NOT NULL,      /* Sno 属性不允许取空值 */
 Cno CHAR(5) NOT NULL,      /* Cno 属性不允许取空值 */
 Grade SMALLINT NOT NULL,    /* Grade 属性不允许取空值 */
 Semester CHAR(5),
 Teachingclass CHAR(8),
 PRIMARY KEY(Sno, Cno),      /* 在表级定义实体完整性,隐含 Sno、Cno 不允许 */
                                /* 取空值,在列级不允许取空值的定义可不写 */
);
```

3. 列值唯一

[例 5.6]建立学院表 School,要求学院名称 SHname 列取值唯一,学院编号 SHno 列为主码。

```
CREATE TABLE School
(SHno CHAR(8) PRIMARY KEY,    /* SHno 列为主码 */
 SHname VARCHAR(40) UNIQUE,   /* 要求 SHname 值唯一 */
 SHfounddate Date             /* 学院创建日期 */
);
```

4. 用 CHECK 短语指定列值应该满足的条件

[例 5.7]Student 表的 Ssex 属性只允许取“男”或“女”。

```
CREATE TABLE Student
(Sno CHAR(8) PRIMARY KEY,      /* 在列级定义主码 */
 Sname CHAR(20) NOT NULL,      /* Sname 属性不允许取空值 */
 Ssex CHAR(6) CHECK(Ssex IN('男','女')), /* 性别属性 Ssex 只允许取'男'或'女' */
 Sbirthdate Date,
```



```
Smajor VARCHAR(40)
);
```

[例 5.8] SC 表 Grade 的值应该在 0~100。

```
CREATE TABLE SC
(Sno CHAR(8),
Cno CHAR(5),
Grade SMALLINT CHECK (Grade >= 0 AND Grade <= 100), /* Grade 取值范围 0~100 */
Semester CHAR(5),
Teachingclass CHAR(8),
PRIMARY KEY(Sno, Cno),
FOREIGN KEY(Sno) REFERENCES Student(Sno),
FOREIGN KEY(Cno) REFERENCES Course(Cno)
);
```

5. 属性上约束的检查和违约处理

当往表中插入元组或修改属性的值时，关系数据库管理系统将检查属性上的约束是否被满足，如果不满足则操作被拒绝执行。

5.4.2 元组上的约束

1. 元组上约束的定义

与属性上约束的定义类似，在 CREATE TABLE 语句中可以用 CHECK 短语定义元组上的约束，即元组级的限制。同属性值限制相比，元组级的限制可以设置不同属性之间取值的相互约束。

[例 5.9] 当学生的性别是“男”时，其姓名不能以 Ms. 打头。

```
CREATE TABLE Student
(Sno CHAR(8),
Sname CHAR(20) NOT NULL,
Ssex CHAR(6),
Sbirthdate Date,
Smajor VARCHAR(40),
PRIMARY KEY(Sno),
CHECK(Ssex='女' OR Sname NOT LIKE 'Ms. %')
); /* 定义了元组中 Sname 和 Ssex 两个属性值之间的约束条件 */
```

性别是女性的元组都能通过该项 CHECK 检查，因为 Ssex='女' 成立；当性别是男性时，要通过检查则姓名一定不能以 'Ms.' 打头，因为 Ssex='男' 时，条件要想为真值，Sname NOT LIKE 'Ms. %' 必须为真值。

2. 元组上约束的检查和违约处理

当往表中插入元组或修改属性的值时，关系数据库管理系统将检查元组上的约束是否被满足，如果不满足则操作被拒绝执行。

5.5 完整性约束命名子句

以上讲解的完整性约束都是在 CREATE TABLE 语句中定义的。SQL 还在 CREATE TABLE 语句中提供了完整性约束命名子句 CONSTRAINT，用来对完整性约束命名，从而可以灵活地增加、删除一个完整性约束。

1. 完整性约束命名子句格式

CONSTRAINT <完整性约束名> <完整性约束>

<完整性约束>包括 NOT NULL、UNIQUE、PRIMARY KEY、FOREIGN KEY、CHECK 短语等。

[例 5.10] 建立“学生”表 Student，要求学号在 10000000 到 29999999 之间，姓名不能取空值，出生日期在 1980 年之后，性别只能是“男”或“女”。

```
CREATE TABLE Student
( Sno CHAR(8)
  CONSTRAINT C1 CHECK(Sno BETWEEN '10000000' AND '29999999'),
  Sname CHAR(20)
  CONSTRAINT C2 NOT NULL,
  Sbirthdate Date
  CONSTRAINT C3 CHECK(Sbirthdate >'1980-1-1'),
  Ssex CHAR(6)
  CONSTRAINT C4 CHECK(Ssex IN('男','女')),
  Smajor VARCHAR(40),
  CONSTRAINT StudentKey PRIMARY KEY(Sno)
);
```

在 Student 表上建立了 5 个约束条件，包括主码约束(命名为“StudentKey”)和 C1、C2、C3、C4 这 4 个列级约束。

[例 5.11] 建立“教师”表 Teacher，要求每个教师的应发工资(每月)不低于 3 000 元。应发工资是工资列 Sal 与扣除项 Deduct 之和。

```
CREATE TABLE Teacher
( Eno CHAR(8) PRIMARY KEY,    /* 在列级定义主码 */
  Ename VARCHAR(20),
```

```

Job CHAR(8),
Sal NUMERIC(7,2),           /* 每月工资 */
Deduct NUMERIC(7,2),        * 每月扣除项 */
Schoolno CHAR(8),           /* 教师所在的学院编号 */
CONSTRAINT TeacherFKey FOREIGN KEY (Schoolno) REFERENCES School (Schoolno),
/* 外码约束(命名为 TeacherFKey) */
CONSTRAINT C1 CHECK (Sal+Deduct >= 3000) /* 应发工资的约束条件 C1 */
);

```

2. 修改表中的完整性限制

可以使用 ALTER TABLE 语句修改表中的完整性约束。

[例 5.12] 去掉例 5.10 Student 表中对出生日期的限制。

```

ALTER TABLE Student
DROP CONSTRAINT C3;

```

[例 5.13] 修改表 Student 中的约束条件, 要求学号改为在 900000 到 999999 之间, 出生日期改为 1985 年之后。

可以先删除原来的约束条件, 再增加新的约束条件。

```

ALTER TABLE Student
DROP CONSTRAINT C1;
ALTER TABLE Student
ADD CONSTRAINT C1 CHECK (Sno BETWEEN '900000' AND '999999');
ALTER TABLE Student
DROP CONSTRAINT C3;
ALTER TABLE Student
ADD CONSTRAINT C3 CHECK (Sbirthdate > '1985-1-1');

```

* 5.6 域的完整性限制

在第 1、2 章中已经讲到, 域是数据库中一个重要概念。一般地, 域是一组具有相同数据类型的值的集合。SQL 支持域的概念, 可以用 CREATE DOMAIN 语句建立一个域并指明域应该满足的完整性约束, 然后就可以用域来定义属性。这样定义的优点是, 数据库中不同的属性可以来自同一个域, 当域的完整性约束改变时只要修改域的定义即可, 不必一一修改域上的各个属性。

[例 5.14] 建立一个性别域, 并声明性别域的取值范围。

```

CREATE DOMAIN GenderDomain CHAR(6)
CHECK (VALUE IN ('男', '女'));

```

这样例 5.10 中对 Ssex 的说明可以改写为:

```
Ssex GenderDomain
```

[例 5.15] 建立一个性别域 GenderDomain, 并对其中的限制命名。

```
CREATE DOMAIN GenderDomain CHAR(6)
CONSTRAINT GD CHECK(VALUE IN ('男','女'));
```

[例 5.16] 删除域 GenderDomain 的限制条件 GD。

```
ALTER DOMAIN GenderDomain
DROP CONSTRAINT GD;
```

[例 5.17] 在域 GenderDomain 上增加性别的限制条件 GDD。

```
ALTER DOMAIN GenderDomain
ADD CONSTRAINT GDD CHECK (VALUE IN('1','0'));
```

这样, 通过例 5.16 和例 5.17, 就把性别的取值范围由('男','女')改为('1','0')。

SQL 标准支持数据库完整性断言 (ASSERTION) 功能。用户可以使用 DDL 中的 CREATE ASSERTION 语句, 通过声明性断言 (declarative assertion) 来指定更具一般性的完整性约束, 可以定义涉及多个表或聚集操作的比较复杂的完整性约束 (详细内容可扫描二维码阅读)。



扩展阅读: SQL
断言

5.7 触 发 器

触发器是用户定义在关系表上的一类由事件驱动的特殊过程。触发器类似于约束, 但是比约束更加灵活, 可以实施更为复杂的检查和违约操作。它不是只提供一种选择, 即中止导致违约的事务/操作, 因此具有更精细和更强大的数据控制能力。

触发器一经定义就被保存在数据库服务器中。任何用户对表的更新操作均由服务器自动激活相应的触发器, 在关系数据库管理系统核心层进行集中的完整性控制。

触发器在 SQL 99 之后才写入 SQL 标准, 但是不少关系数据库管理系统早就支持触发器, 因此不同的关系数据库管理系统实现的触发器语法各不相同、互不兼容。本节的例子是按照 SQL 标准写的, 不同的关系数据库管理系统产品会有差别, 请读者在上机实验时注意阅读所安装系统的使用说明。

5.7.1 定义触发器

触发器又叫作事件-条件-动作规则 (event-condition-action rule, ECA rule)。当特定的系

统事件(如对一个表的更新操作,事务的结束等)发生时,对规则的条件进行检查,如果条件成立则执行规则中的动作,否则不执行该动作。规则中的动作体可以很复杂,涉及其他表和其他数据库对象,通常是一段 SQL 存储过程。

SQL 使用 CREATE TRIGGER 命令建立触发器,其一般格式如下:

```
CREATE TRIGGER <触发器名>                                /* 每当触发事件发生时,该触发器被激活 */
[BEFORE | AFTER] <触发事件> ON <表名>                  /* 指明触发器激活的时间是在执行
                                                         /* 触发事件前或后 */
REFERENCING NEW|OLD AS<变量>                          /* REFERENCING 指出引用的变量 */
FOR EACH[ROW | STATEMENT]                               /* 定义触发器的类型,指明动作体执行的频率 */
[WHEN <触发条件>]<触发动作体>                         /* 仅当触发条件为真时才执行触发动作体 */
```

下面对定义触发器的各部分语法进行详细说明。

① 创建触发器。只有表的拥有者,即创建表的用户才可以在表上创建触发器,并且一个表上只能创建一定数量的触发器。触发器的具体数量由具体的关系数据库管理系统产品在设计时确定。

② 触发器名。触发器名可以包含模式名,也可以不包含模式名。同一模式下的触发器名必须是唯一的,并且触发器名和表名必须在同一模式下。

③ 表名。触发器只能定义在基本表上,不能定义在视图上。当基本表的数据发生变化时,将激活定义在该表上相应触发事件的触发器,因此该表也称为触发器的目标表。

④ 触发事件。触发事件可以是 INSERT、DELETE 或 UPDATE,也可以是这几个事件的组合,如 INSERT OR DELETE、INSERT AND DELETE 等,还可以是 UPDATE OF <触发列, ...>,即进一步指明修改哪些列时激活触发器。

AFTER/BEFORE 是触发的时机。AFTER 表示在触发事件的操作执行之后激活触发器, BEFORE 则表示在触发事件的操作执行之前激活触发器。

⑤ 触发器类型。触发器按照所触发动作的间隔尺寸可以分为行级触发器(FOR EACH ROW)和语句级触发器(FOR EACH STATEMENT)。

如果定义的触发器为语句级触发器,那么执行了此 UPDATE 语句后触发动作体将执行一次。如果是行级触发器,那么 UPDATE 语句影响多少行,就触发多少次。

⑥ 触发条件。触发器被激活时,只有当触发条件为真时触发动作体才执行,否则触发动作体不执行。如果省略 WHEN 触发条件,则触发动作体在触发器激活后立即执行。

⑦ 触发动作体。触发动作体既可以是一个匿名 PL/SQL 过程块,也可以是对已创建存储过程的调用。如果是行级触发器,用户可以在过程体中使用 NEW 或 OLD 引用 UPDATE/INSERT 事件之后(前)的新(旧)值;如果是语句级触发器,则不能在触发动作体中使用 NEW 或 OLD 进行引用。

如果触发动作体执行失败,激活触发器的事件(即对数据库的更新操作)就会终止执行,触发器的目标表或触发器可能影响的其他对象不发生任何变化。

[例 5.18] 当对表 SC 的 Grade 属性进行修改时, 若分数增加了 10%, 则将此次操作记录到另一个表 SC_U(Sno CHAR(8), Cno CHAR(5), Oldgrade SMALLINT, Newgrade SMALLINT) 中。其中, Oldgrade 是修改前的分数, Newgrade 是修改后的分数。运行触发器之前需要创建 SC_U 表。

```
CREATE TRIGGER SC_T          /* SC_T 是触发器的名字 */
AFTER UPDATE ON SC          /* UPDATE ON SC 是触发事件, */
    /* AFTER 是触发的时机, 表示当对 SC 的 Grade 属性修改完后再触发下面的规则 */
REFERENCING
    OLD AS OldTuple
    NEW AS NewTuple
FOR EACH ROW /* 行级触发器, 即每执行一次 Grade 的更新, 下面的规则就执行一次 */
WHEN (NewTuple. Grade >= 1.1 * OldTuple. Grade)
    /* 触发条件, 只有该条件为真时才执行下面的 insert 操作 */
BEGIN
    INSERT INTO SC_U (Sno, Cno, OldGrade, NewGrade) /* 触发动作体 */
    VALUES(OldTuple. Sno, OldTuple. Cno, OldTuple. Grade, NewTuple. Grade)
END
```

在本例中 REFERENCING 指出引用的变量, 如果触发事件是 UPDATE 操作并且有 FOR EACH ROW 子句, 则可以引用的变量有 OLD 和 NEW, 分别表示修改之前的元组和修改之后的元组。若没有 FOR EACH ROW 子句, 则可以引用的变量有 OLD TABLE 和 NEW TABLE, OLD TABLE 表示表中原来的内容, NEW TABLE 表示表中变化后的部分。

[例 5.19] 将每次对表 Student 的插入操作所增加的学生个数记录到表 Student InsertLog (numbers INT) 中, 运行触发器之前需要创建此表。

```
CREATE TRIGGER Student_Count
AFTER INSERT ON Student /* 指明触发器激活的时间是在执行 INSERT 后 */
REFERENCING
    NEW TABLE AS Delta
FOR EACH STATEMENT /* 语句级触发器, 即执行完 INSERT 语句后 */
    /* 下面的触发动作体才执行一次 */
BEGIN
    INSERT INTO StudentInsertLog (Numbers)
    SELECT COUNT(*) FROM Delta
END
```

在本例中出现的“FOR EACH STATEMENT”表示触发事件 INSERT 语句执行完成后才执行一次触发器中的动作, 这种触发器叫作语句级触发器。而例 5.18 中的触发器是行级触发器。默认的触发器是语句级触发器。“Delta”是一个关系名, 其模式与 Student 相同, 包含的元组是

INSERT 语句增加的元组。

[例 5.20] 定义一个 BEFORE 行级触发器, 为教师表 Teacher 定义完整性规则: 教授的工资不得低于 4 000 元, 如果低于 4 000 元, 自动改为 4 000 元。

```
CREATE TRIGGER Update_Sal      /* 对教师表插入或更新时激活触发器 */
BEFORE UPDATE ON Teacher      /* BEFORE 触发事件 */
REFERENCING NEW AS newTuple
FOR EACH ROW                  /* 这是行级触发器 */
BEGIN                          /* 定义触发动作体, 这是一个 PL/SQL 过程块 */
    IF (newTuple.job='教授') AND (newTuple.sal < 4000)
        /* 因为是行级触发器, 可在过程体中 */
        THEN newTuple.sal := 4000; /* 使用插入或更新操作后的新值 */
    END IF;
END;                            /* 触发动作体结束 */
```

因为定义的是 BEFORE 触发器, 在插入和更新教师记录前就可以按照触发器的规则调整教授的工资, 不必等插入后再检查调整。

5.7.2 执行触发器

触发器的执行是由触发事件激活, 并由数据库服务器自动执行的。一个数据表上可能定义了多个触发器, 如多个 BEFORE 触发器、多个 AFTER 触发器等, 同一个表上的多个触发器激活时遵循如下的执行顺序:

- ① 执行该表上的 BEFORE 触发器。
- ② 激活触发器的 SQL 语句。
- ③ 执行该表上的 AFTER 触发器。

对于同一个表上的多个 BEFORE/AFTER 触发器, 遵循“谁先创建谁先执行”的原则, 即按照触发器创建的时间先后顺序执行。有些关系数据库管理系统是按照触发器名称的字母排序顺序执行触发器。

5.7.3 删除触发器

删除触发器的 SQL 语句格式如下:

DROP TRIGGER <触发器名> ON <表名>;

触发器必须是一个已经创建的触发器, 并且只能由具有相应权限的用户删除。

触发器是一种功能强大的工具, 但在使用时要慎重, 因为在每次访问一个表时都可能触发一个触发器, 这样会影响系统性能。

本章小结

本章系统讲解了数据库完整性的概念,介绍了使用 SQL 语句实现数据库完整性定义的方法。关系数据库管理系统将负责检查完整性约束,当发现违约时进行处理。对于违反参照完整性的操作,本书讲解了不同的处理策略。用户要根据应用语义来定义合适的处理策略,以保证数据库的正确性。

本章还介绍了实现数据库完整性的方法之一——触发器。

触发器与各种完整性约束方法相比,其违约处理的功能更加灵活,更加强大。当违约时,触发器的动作体可以是一段 SQL 存储过程或 PL/SQL 过程块。触发器不仅可以用于数据库完整性检查,也可以用来实现数据库系统的其他功能,包括数据库安全性、应用系统的一些业务流程和控制流程等。不过也要注意,一个触发器的动作可能激活另一个触发器,最坏的情况是导致激活一个触发链,从而有可能造成难以预见的错误。

习题 5

1. 什么是数据库的完整性?
2. 数据库的完整性概念与数据库的安全性概念有什么区别和联系?
3. 什么是数据库的完整性约束?
4. 关系数据库管理系统的完整性控制机制应具备哪三方面的功能?
5. 关系数据库管理系统在实现参照完整性时需要考虑哪些方面?
6. 假设下面两个关系模式:

职工(职工号,姓名,出生日期,职务,工资,部门号),其中职工号为主码;
部门(部门号,名称,经理姓名,电话),其中部门号为主码。

用 SQL 定义这两个关系模式,要求在模式中完成以下完整性约束的定义:

- ① 定义每个模式的主码。
- ② 定义参照完整性约束。
- ③ 定义职工年龄不超过 65 岁。

7. 在关系数据库管理系统中,当操作违反实体完整性、参照完整性和用户定义的完整性约束时,一般是如何分别进行处理的?

* 8. 某单位想举行一个小型联谊会,关系 Male 记录注册的男宾信息,关系 Female 记录注册的女宾信息。建立一个断言,将来宾的人数限制在 50 人以内(提示:先创建关系 Female 和关系 Male)。

第5章实验 完整性控制

实验 5.1 实体完整性实验。掌握实体完整性的定义和维护方法。定义实体完整性,删除实体完整性。能够写出两种方式定义实体完整性的 SQL 语句:创建表时定义实体完整性、创建表后使用 ALTER TABLE 语句定义实体完整性。设计 SQL 语句验证完整性约束是否起作用。

实验 5.2 参照完整性实验。掌握参照完整性的定义和维护方法。定义参照完整性,定义参照完整性的违

约处理，删除参照完整性。能够写出两种方式定义参照完整性的 SQL 语句：创建表时定义参照完整性、创建表后使用 ALTER TABLE 语句定义参照完整性。

实验 5.3 用户自定义完整性实验。掌握用户自定义完整性的定义和维护方法。针对具体应用语义，选择 NULL/NOT NULL、DEFAULT、UNIQUE、CHECK 等定义属性上的约束条件。

实验 5.4 触发器实验。掌握数据库触发器的设计和使用方法。定义 BEFORE 触发器和 AFTER 触发器、行级触发器或语句级触发器。能够理解不同类型触发器的作用和执行原理，设计数据操纵语言(DML)语句验证触发器的有效性。

参考文献 5

[1] HAMMER M M, MCLEOD D J. Semantic integrity in a relational data base system[C]. Proceedings of the 1st International Conference on Very Large Data Bases, 1975: 25-47.

文献[1]讨论数据库管理系统中语义完整性的概念。

[2] CODD E F. Extending the data base relational model to capture more meaning[J]. ACM Transactions on Database Systems, 1979, 4(4): 397-434.

文献[2]扩展了关系模型的语义表达能力。

[3] STONEBRAKER M. Implementation of integrity constraints and views by query modification[C]. Proceedings of International Conference on ACM SIGMOD, 1975.

文献[3]讨论 INGRES 数据库管理系统的完整性约束机制。

[4] CHAMBERLIN D D, ASTRAHAN M M, ESWARAN K P, et al. Sequel 2: a unified approach to data definition, manipulation, and control[J]. IBM Journal of Research and Development, 1976, 20(6): 560-575.

文献[4]提出了 SYSTEM R 中 SQL 的 Integrity Assert(完整性断言)和触发器语句。

[5] BERNSTEIN P A, BLAUSTEIN B T, CLARKE E M. Fast maintenance of semantic integrity assertions using redundant aggregate data[C]. Proceedings of the 6th International Conference on Very Large Data Bases, 1980, 6: 126-136.

[6] HSU A, IMIELINSKI T. Integrity checking for multiple updates[J]. ACM SIGMOD Record, 1985, 14(4): 152-168.

本章知识点讲解微视频:



模型固有的完整性



用户定义的完整性



5-3 通用工具——

触发器

第二篇

设计与应用开发篇

本篇讲解在开发应用系统时如何设计数据库，以及如何基于已经选定的数据库管理系统进行编程。

本篇包括三章。

第6章关系数据理论，详细讲解关系规范化理论，它既是关系数据库的重要理论基础，也是数据库设计的有力工具。规范化理论为数据库设计提供了理论指南和工具。

第7章数据库设计，讨论数据库设计的方法和步骤，详细讲解数据库设计各阶段的目标、方法和应注意的问题；重点讲解概念结构和逻辑结构的设计。以“高校本科教务管理”信息系统为示例，抽象出“学生选课管理”“学生学籍管理”“教师教学管理”三个应用场景，阐述如何在理论的指导下完成 E-R 图的设计、集成和优化。

第8章数据库编程，从两方面讲解数据库编程技术和方法：一是使用扩展的 SQL 功能，包括引入新的 SQL 子句、新的数据库内置函数、PL/SQL 与存储过程/自定义函数；二是在高级语言程序中使用 SQL，实现高级语言与数据库之间的交互，开发应用需求中复杂业务逻辑的技术和方法。本章还介绍了 JDBC 编程的原理、方法和步骤。

关系数据理论的主要内容是关系规范化理论,用来指导数据库逻辑设计,具体包括两部分:一是给出判断数据库逻辑设计“好坏程度”的准则;二是如果逻辑设计中存在“不好”的关系模式,如何将其修改为“好”的关系模式。

本章导读

在关系数据库模式设计中,我们面临两个问题,第一,什么是一个“好”的关系数据库模式,其标准是什么。如果没有判断的标准,就很难对一个具体的设计进行评价。第二,能否开发数据库设计辅助工具来减轻设计者的负担。本章6.1和6.2节介绍关系规范化理论,用于回答第一个问题;6.3和6.4节介绍函数依赖公理系统以及模式分解算法,探索自动进行模式分解的方法,用于回答第二个问题。

本章首先从数据库逻辑设计中如何构造一个好的数据库模式这一问题出发,阐明关系规范化理论研究的实际背景,然后讨论两类最重要的数据依赖——函数依赖和多值依赖。读者要牢固掌握数据依赖的基本概念,如函数依赖、部分函数依赖、完全函数依赖、传递函数依赖等,掌握1NF到4NF的概念、规范化的含义和作用;能够了解各种范式以及在插入、删除和更新数据时可能出现的问题,并直观地描述解决办法。

本章进一步讨论函数依赖的推理规则,介绍函数依赖公理系统,讲解保持函数依赖的模式等价准则及相应的模式分解算法。要掌握这些算法并在数据库模式设计中应用。6.5节是选读内容,介绍无损连接的模式分解概念及相应算法,并解释其与保持函数依赖的模式分解之间的关系。

读者学习本章后应能够根据应用语义完整地写出关系模式的数据依赖集合,并能根据数据依赖分析某一个关系模式属于第几范式,能够运用介绍的有关算法辅助设计和优化数据库模式。

6.1 问题的提出

前面已经讨论了数据库系统的一般概念,介绍了关系数据库的基本概念、关系模型的三要